RV College of Engineering
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Non-Human Identity Governance Agent

MAJOR PROJECT REPORT
(IS481P)

Submitted by,

Manu Prakash Bhat **1RV22IS035**

Under the guidance of

Dr. Merin Meleet
Associate Professor
Dept. of ISE
RV College of Engineering

Mr. Tejeshwar Rao
Manager DevSecOps & Cloud
DevSecOps & Cloud Engineering
Honeywell Technology Solutions

In partial fulfillment for the award of degree of

Bachelor of Engineering
in
Information Science and Engineering

Department of ISE
2025-26

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Non-Human Identity Governance Agent

A Major Project Report (IS481P)

Submitted by,

Manu Prakash Bhat **1RV22IS035**

Under the guidance of

Dr. Merin Meleet
Associate Professor
Dept. of ISE
RV College of Engineering

Mr. Tejeshwar Rao
Manager DevSecOps & Cloud
DevSecOps & Cloud Engineering
Honeywell Technology Solutions

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Information Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Information Science and Engineering

CERTIFICATE

Certified that the Major Project (IS481P) work titled ***Non-Human Identity Governance Agent*** is carried out by **Manu Prakash Bhat** (**1RV22IS035**) who is bonafide student of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	Head of the Department	Principal
Dr. Merin Meleet	Dr. G S Mamatha	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

I, **Manu Prakash Bhat** student of eighth semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Non-Human Identity Governance Agent**' has been carried out by me and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Information Science and Engineering** during the year 2025-26.

Further I declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

I also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and I will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Manu Prakash Bhat(1RV22IS035)	

ACKNOWLEDGEMENTS

I am indebted to my guide, **Dr. Merin Meleet**, Associate Professor, Department of ISE, RVCE for the wholehearted support, suggestions and invaluable advice throughout my project work and also helped in the preparation of this thesis.

My sincere thanks to the project coordinator **Dr. Vanishree K**, Associate Professor, Department of ISE for the timely instructions and support in coordinating the project.

My gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

My sincere thanks to **Dr. G S Mamatha**, Professor and Head, Department of Information Science and Engineering, RVCE for the support and encouragement.

I express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

I thank all the teaching staff and technical staff of Information Science and Engineering department, RVCE for their help.

Lastly, I take this opportunity to thank my family members and friends who provided all the backup support throughout the project work.

ABSTRACT

The proliferation of cloud-native architectures and DevOps automation has resulted in an exponential growth of Non-Human Identities (NHIs)—service principals, API tokens, deploy keys, certificates, and managed identities—which now outnumber human users by ratios exceeding 45:1 in enterprise environments. According to the OWASP Non-Human Identities Top 10 (2025), approximately 70% of organisations maintain plaintext secrets in code repositories, and credential-based attacks remain a primary breach vector responsible for significant data exposure incidents. Despite the critical nature of this threat surface, existing identity governance solutions remain fragmented: vault-based tools manage known secrets but lack discovery capabilities, while platform-native solutions operate in isolation without cross-environment visibility.

This project addresses the governance gap through the design and development of an autonomous Non-Human Identity Governance Agent—a multi-layer system that discovers, classifies, scores, and remediates machine credentials across heterogeneous cloud platforms. The system employs a modular agent-based architecture comprising discovery agents for multiple credential types, a deterministic risk scoring engine operating on a 100-point composite scale, policy-as-code enforcement through dual Python and OPA Rego engines, unsupervised machine learning via Isolation Forest for anomaly detection, and linear regression for temporal trend prediction. Automated remediation capabilities enable cryptographic credential rotation through Ed25519 key generation and sealed-box encryption.

The developed system demonstrates cross-platform NHI discovery across GitHub and Microsoft Azure environments, AI-driven risk quantification, policy compliance evaluation against SOC 2 Type II, NIST Cybersecurity Framework, ISO 27001, and CIS Benchmarks, and real-time monitoring through an interactive dashboard. The approach bridges the gap between secret management and identity governance by providing unified visibility, intelligent risk prioritisation, and actionable remediation for the growing non-human identity attack surface.

Keywords: Non-Human Identities, Identity Governance, DevSecOps, Machine Learning, Anomaly Detection, Policy-as-Code, Zero Trust Architecture, Cloud Security

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

API Application Programming Interface

CI/CD Continuous Integration / Continuous Deployment

Ed25519 Edwards-curve Digital Signature Algorithm (Curve 25519)

GA GitHub Actions

IAM Identity and Access Management

ISO International Organization for Standardization

ML Machine Learning

NHI Non-Human Identity

NIST National Institute of Standards and Technology

OPA Open Policy Agent

OWASP Open Worldwide Application Security Project

PAT Personal Access Token

REST Representational State Transfer

SDK Software Development Kit

SOC System and Organisation Controls

SP Service Principal

SSH Secure Shell

SSL Secure Sockets Layer

TLS Transport Layer Security

ZTA Zero Trust Architecture

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Machine credentials in enterprise environments are poorly governed despite their sheer volume and the access they hold. This chapter presents the context and motivation for the Non-Human Identity Governance Agent, defines the problem it addresses, states the project objectives and methodology, and outlines how this report is organised.

1.1 Introduction

The term Non-Human Identity refers to any credential, token, certificate, or service account that enables machine-to-machine communication without direct human involvement. In contemporary cloud-native environments, these identities manifest as Application Programming Interface (API) keys, Personal Access Tokens (PATs), deploy tokens, Service Principals (SPs), managed identities, Secure Shell (SSH) keys, and Transport Layer Security (TLS) certificates. Their numbers grow with every new microservice, Continuous Integration / Continuous Deployment (CI/CD) pipeline, and infrastructure-as-code deployment.

The problem is scale combined with neglect. Enterprise environments routinely maintain tens of thousands of machine credentials, many of which hold elevated privileges, lack rotation schedules, and persist long after the workloads they were created for have been decommissioned. A dormant service principal is a far easier target for an attacker than a human account protected by multi-factor authentication and behavioural monitoring.

The Open Worldwide Application Security Project (OWASP) Non-Human Identities Top 10 report published in 2025 brought formal recognition to this category of risk, identifying ten critical vulnerability patterns including improper credential offboarding, secret leakage in repositories, and overprivileged service accounts [1]. The report found that seven out of ten organisations maintain plaintext secrets in their source code repositories. This statistic alone illustrates the gap between the security investment directed at human identities and the comparatively minimal governance applied to their machine counterparts.

1.2 Literature Review

Research on the non-human identity threat landscape consistently identifies credential sprawl as one of the most underaddressed risk surfaces in enterprise security. In-

dustry analyses show that Non-Human Identitys (NHIs) outnumber human identities at ratios ranging from 17:1 to over 100:1 in large organisations, with annual growth rates approaching 44% [2], [3]. The Verizon Data Breach Investigations Report identifies credential abuse as the primary attack vector in nearly half of all data breaches globally [4]. Surveys of security practitioners confirm the operational gap: only 15% of organisations report high confidence in preventing NHI-related attacks, and 69% express concern about their machine identity posture [5]. Threat intelligence frameworks document how adversaries specifically target long-lived, unrotated service credentials as a reliable means of establishing persistent access [6]. Research on software supply chain security further establishes that hardcoded secrets in source repositories remain a primary initial access vector in cloud-targeted attacks [7], [8].

The zero trust security model provides the theoretical framework for addressing the NHI governance problem. National Institute of Standards and Technology (NIST) Special Publication 800-207 defines the zero trust architecture and mandates continuous verification, least-privilege enforcement, and assumed-breach posture for all entities including non-human ones [9]. Research on Identity and Access Management (IAM) in cloud environments identifies the tension between automated credential provisioning and continuous verification requirements [10], [11]. Cross-cloud federated IAM systems using trust score computation have been proposed to enforce least-privilege access across cloud boundaries [12]. The emergence of autonomous AI agents introduces additional governance challenges, with research demonstrating the inadequacy of conventional authentication protocols for agentic systems [13]. Compliance frameworks including System and Organisation Controls (SOC) 2, International Organization for Standardization (ISO) 27001, and the NIST Cybersecurity Framework all mandate formal credential lifecycle controls that apply equally to machine identities [14], [15], [16].

Machine learning applied to identity and access management has demonstrated value in detecting anomalous behaviour that rule-based systems miss. Surveys of anomaly detection methodologies categorise approaches into statistical, clustering-based, and information-theoretic methods, with unsupervised algorithms proving effective for security datasets where labelled examples of attack behaviour are scarce [17]. The Isolation Forest algorithm isolates anomalies through random partitioning and has been shown to be efficient for high-dimensional access pattern datasets [18]. Research on AI-driven dynamic trust

assessment proposes frameworks using both supervised and unsupervised Machine Learning (ML) to compute continuous trust scores for identity verification, moving beyond binary authenticated-or-not decisions [19]. The broader landscape of ML applied to security operations documents both the promise of these techniques and practical deployment challenges including false positive management [20].

Policy-as-code approaches replace manual, document-based governance with machine-executable policy definitions that integrate into CI/CD pipelines. The Open Policy Agent with its Rego language has emerged as the leading framework for declarative governance enforcement, enabling version-controlled, testable compliance rules [21], [22]. Literature reviews on software secret management document the pervasive challenge of credential sprawl and demonstrate that automated rotation can reduce exposure windows from months to hours [23], [24]. DevSecOps research confirms that organisations integrating automated security controls into development workflows achieve faster remediation and measurably stronger security posture [25], [26]. Certificate lifecycle management studies highlight the operational consequences of disconnected expiry monitoring, with unplanned outages from expired certificates representing a recurring industry problem [27].

Existing commercial tools address individual components of the NHI governance problem but none integrates the full lifecycle autonomously. HashiCorp Vault provides mature secrets management and dynamic credential generation but does not discover NHIs across multi-platform environments or perform risk scoring [28]. Microsoft Entra Workload ID offers Azure-native identity services with managed identity support but lacks cross-platform visibility and AI-driven classification [29]. The absence of a unified tool combining autonomous discovery, AI-driven risk quantification, policy-as-code enforcement, and automated remediation across heterogeneous cloud platforms constitutes the research gap that this project addresses.

1.3 Motivation

Several factors motivated the development of an automated governance agent for non-human identities. First, scale: NHIs outnumber human identities at ratios of 17:1 in moderately sized organisations and over 45:1 in large enterprises with extensive automation [2], [3]. Managing this volume of credentials manually is not feasible, yet most organisations lack any automated tooling for NHI lifecycle management.

Second, threat severity: credential-based attacks account for a large proportion of data

breaches. The Verizon Data Breach Investigations Report identified credential abuse as the primary attack vector in nearly half of all recorded incidents [4]. When adversaries obtain a long-lived, unrotated service principal with broad permissions, they gain persistent access that is hard to detect because the credential usage appears legitimate.

Third, the regulatory landscape: compliance frameworks including SOC 2, ISO 27001, and NIST Cybersecurity Framework all mandate controls for credential lifecycle management, access reviews, and least-privilege enforcement [9], [14], [15]. Organisations that cannot demonstrate automated governance of their machine credentials face audit findings and regulatory consequences. The absence of a unified tool combining discovery, scoring, policy enforcement, and remediation across cloud platforms leaves both a security gap and a compliance gap.

Observations during an internship at Honeywell Technology Solutions confirmed how large engineering teams struggle with credential sprawl across GitHub repositories and Azure subscriptions. Service principals created for proof-of-concept projects remained active months after the projects concluded. Deploy keys granted write access persisted unchanged across multiple team rotations. These observations confirmed that the problem is not theoretical but a daily operational reality in enterprise environments.

1.4 Problem Statement

Enterprise organisations deploying workloads across GitHub and Azure cloud ecosystems lack a unified, autonomous system for continuously discovering non-human identities, quantifying their risk posture, enforcing governance policies, and executing automated remediation. Existing tools address individual aspects of the credential lifecycle in isolation, requiring manual coordination between discovery, assessment, and rotation processes. This fragmented approach results in unmanaged credentials persisting beyond their useful life, accumulating excessive privileges, and escaping the periodic access reviews that human accounts receive. The project addresses this gap by implementing an integrated governance agent that operates across the full NHI lifecycle from discovery through remediation.

1.5 Objectives

The objectives of this project are:

1. Design and implement an autonomous agent that continuously discovers non-human

identities across GitHub repositories and Azure cloud services (personal access tokens, deploy keys, Actions secrets, service principals, managed identities, Key Vault secrets, and TLS certificates).

2. Quantify the risk of each discovered identity through a deterministic 100-point scoring algorithm combining credential type, age, privilege scope, and dormancy indicators, augmented by Isolation Forest anomaly detection and linear regression trend prediction.

3. Enforce codified governance policies through eleven rules implemented in both Python and Open Policy Agent (OPA) Rego, with transparent fallback when one engine is unavailable.

4. Automate credential rotation for deploy keys and Actions secrets using Edwards-curve Digital Signature Algorithm (Curve 25519) (Ed25519) key generation and sealed-box encryption.

5. Deliver a real-time monitoring dashboard (React with TypeScript) giving security operators interactive visibility into the NHI inventory, risk trends, policy violations, and compliance posture.

1.6 Brief Methodology of the Project

The project followed an iterative development methodology structured across three phases. The first phase established the discovery and monitoring foundation, implementing Secure Sockets Layer (SSL)/TLS certificate checking, GitHub NHI enumeration, and Azure service principal scanning. The second phase introduced the intelligence layer with risk scoring, policy enforcement via OPA, and the machine learning analysis engine. The third phase delivered the remediation capabilities and the interactive React dashboard for operational use.

Each phase followed a cycle of requirements analysis, component implementation, integration testing, and refinement based on real-world scanning results against the target enterprise environment. The architecture was designed as a set of loosely coupled Python agents coordinated by a central orchestrator, enabling independent development and testing of each discovery module before integration into the full pipeline. The choice of Python 3.11 provided mature async capabilities and extensive library support for the

required cloud Software Development Kit (SDK) integrations, while React 19 with TypeScript offered type safety and component-based architecture for the frontend monitoring interface.

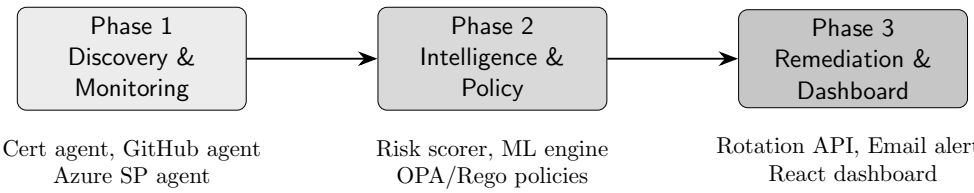


Figure 1.1: Three-phase development methodology

The methodology diagram in Figure 1.1 illustrates the progression from foundational discovery through intelligence and finally to operational remediation and monitoring capabilities.

1.7 Assumptions made / Constraints of the Project

The system operates under several assumptions about the deployment environment. It assumes the availability of a GitHub PAT with sufficient read scopes to enumerate repositories, deploy keys, and Actions secrets for the target organisation. Azure scanning requires service principal credentials with Graph API read access for service principal enumeration and Key Vault reader permissions for secret and certificate discovery. Network connectivity to the target cloud environments must be available from the execution host, and the Python 3.11 runtime with all dependencies specified in the requirements file must be installed.

The primary constraints of the project relate to the boundaries of automated remediation. PAT rotation cannot be performed programmatically through the GitHub API and therefore requires manual operator action guided by the system recommendations. The dashboard operates without built-in authentication, assuming deployment behind a corporate reverse proxy or within a trusted network boundary. The ML analysis requires a minimum of three historical scan snapshots before trend prediction becomes meaningful, which means the first several runs of the system produce limited predictive output. Finally, the OPA binary is an optional dependency; when unavailable, the system transparently falls back to the equivalent Python policy engine without loss of enforcement coverage.

1.8 Organization of the Report

The remainder of this report is organised into six chapters followed by appendices. Chapter 2 presents the theoretical foundations and core concepts including NHI taxonomy, zero trust architecture, risk quantification theory, the Isolation Forest algorithm, policy-as-code paradigms, cryptographic primitives for rotation, and compliance framework requirements. Chapter 3 details the system architecture and component design across the four-layer modular structure, including the dashboard architecture, security design principles, and data flow specifications. Chapter 4 describes the implementation of each layer, covering the development environment, agent implementations, orchestrator logic, and React frontend construction. Chapter 5 presents the testing methodology, unit test coverage, performance benchmarks, security testing practices, and integration testing results. Chapter 6 presents the results and discussion, including security analysis findings, machine learning evaluation metrics, dashboard validation, and a comparative assessment against existing tools. Chapter 7 concludes with a summary of achievements, outlines future work for extending the system, and reflects on the learning outcomes gained through the project.

Summary

This chapter introduced the non-human identity governance problem, reviewed the literature on the NHI threat landscape, zero trust governance, AI-driven anomaly detection, policy-as-code approaches, and identified the research gap addressed by this project. The project objectives, development methodology, and report organisation were defined. Chapter 2 presents the theoretical foundations and core technical concepts that underpin the system design.

Chapter 2

Theory and Fundamentals

CHAPTER 2

THEORY AND FUNDAMENTALS

This chapter presents the theoretical foundations and core concepts underlying the Non-Human Identity Governance Agent. It covers the NHI taxonomy, zero trust architecture principles, risk quantification theory, machine learning algorithms for anomaly detection, policy-as-code paradigms, cryptographic primitives for credential rotation, and the compliance frameworks against which governance posture is evaluated.

2.1 Non-Human Identity Taxonomy and Lifecycle

A Non-Human Identity (NHI) is any credential, token, certificate, or service account that enables machine-to-machine communication without direct human involvement. In cloud-native environments, NHIs manifest across several distinct categories, each with characteristic risk profiles and lifecycle properties [1], [2].

Personal Access Tokens (PATs) are long-lived bearer tokens that grant programmatic access to platform APIs. They inherit the permissions of the issuing human user and persist until explicitly revoked. Two subtypes exist: classic tokens (coarse-grained scopes) and fine-grained tokens (repository-scoped, time-limited). Classic PATs with write permissions represent the highest-risk NHI category due to their broad access and indefinite validity [30].

Deploy Keys are SSH public keys attached to specific repositories, granting either read-only or read-write access. Unlike PATs, they are repository-scoped and do not inherit user-level permissions. However, write-enabled deploy keys that persist beyond their operational need create lateral movement opportunities [3].

Service Principals and Managed Identities are Azure Active Directory objects representing applications or services. Service principals carry client secrets or certificates for authentication, while managed identities eliminate stored credentials through Azure-managed token issuance. Service principals with stale credentials pose risk when the associated application is decommissioned but the identity remains active [5].

Actions Secrets and CI/CD Credentials are environment variables injected into automation pipelines during execution. They enable workflows to authenticate against external services but are opaque to auditing once stored—their values cannot be retrieved, only overwritten or deleted [31].

TLS/SSL Certificates provide mutual authentication and transport encryption between services. Certificate expiry failures cause service outages, while compromised certificates enable man-in-the-middle attacks. Certificate lifecycle management requires continuous monitoring of validity periods across all endpoints [27].

The NHI lifecycle consists of five stages: *creation* (provisioning with initial scope), *distribution* (deployment to consuming services), *active use* (ongoing authentication operations), *dormancy* (cessation of use without revocation), and *decommissioning* (revocation and cleanup). Governance failures typically occur at the dormancy-to-decommissioning transition, where credentials persist in an exploitable state after their functional purpose has ended [23].

2.2 Zero Trust Architecture Principles

The zero trust security model replaces perimeter-based defences with the assumption that no entity—human or machine—should receive implicit trust [9]. Three foundational tenets define the model:

1. **Verify explicitly:** Every access request must be authenticated and authorised based on all available data points including identity, location, device health, and resource sensitivity.

2. **Enforce least privilege:** Access rights must be limited to the minimum necessary for the specific task, with just-in-time and just-enough-access principles applied.

3. **Assume breach:** The system must operate under the assumption that the network is already compromised, minimising blast radius through segmentation and continuous verification.

Applied to non-human identities, zero trust mandates that machine credentials receive the same governance rigour as human accounts. This includes continuous verification of credential validity, regular privilege reviews, behavioural monitoring for anomalous access patterns, and automated revocation when trust conditions are violated [10], [13].

The trust assessment for NHIs differs from human identity verification in critical ways. Machine credentials cannot perform interactive multi-factor authentication, do not exhibit behavioural biometrics, and operate at speeds that make real-time human oversight infeasible. Consequently, NHI governance must rely on deterministic policy

evaluation and statistical anomaly detection rather than interactive verification challenges [19].

2.3 Risk Quantification and Composite Scoring

Quantitative risk assessment assigns numerical values to security posture, enabling prioritisation of remediation efforts and objective tracking of governance improvement over time. For NHI governance, a composite scoring model combines multiple risk indicators into a single normalised value [17].

The deterministic risk score R for an identity i is computed as a weighted sum of four components:

$$R_i = \min\left(100,\ w_t \cdot T_i + w_a \cdot A_i + w_p \cdot P_i + w_d \cdot D_i\right) \tag{2.1}$$

where:

- T_i is the **type weight** (maximum 30 points), reflecting the inherent risk profile of the credential category. Classic PATs with write scopes receive maximum weight; managed identities receive minimum weight.
- A_i is the **age component** (maximum 40 points), computed as $\min\left(40, \left\lfloor 40 \cdot \frac{a_i}{a_{\max}} \right\rfloor\right)$ where a_i is the credential age in days and $a_{\max}$ is the policy-defined maximum age for that credential type.
- P_i is the **privilege scope** (maximum 20 points), quantifying the breadth of permissions. Write scopes, administrative access, and cross-repository permissions increase this component.
- D_i is the **dormancy indicator** (15 points if dormant, 0 otherwise), flagging credentials with no recorded usage within the configured monitoring window.

The resulting score maps to severity bands: *Critical* ($R \geq 80$), *High* ($60 \leq R < 80$), *Medium* ($40 \leq R < 60$), and *Low* ($R < 40$). This stratification ensures that remediation resources are directed toward the highest-impact credentials first.

The scoring model is deterministic—given identical inputs, it produces identical outputs—enabling reproducible audits and consistent policy enforcement. This property distinguishes it from probabilistic risk models and ensures that compliance assessments remain stable between evaluation cycles [32].

2.4 Isolation Forest for Anomaly Detection

The Isolation Forest algorithm detects anomalies by exploiting the structural property that outliers are more susceptible to isolation than normal observations [18]. Unlike distance-based or density-based methods, Isolation Forest measures anomaly through the number of random partitions required to isolate a data point.

Algorithm Principle. Given a dataset $X = \{x_1, x_2, \ldots, x_n\}$ with d features, the algorithm constructs an ensemble of t isolation trees ($iTree$). Each tree recursively partitions the data by randomly selecting a feature q and a split value p drawn uniformly from the range $[x^q_{\min}, x^q_{\max}]$. The partitioning continues until every point is isolated or a maximum tree height $l = \lceil \log_2 n \rceil$ is reached.

Path Length and Anomaly Score. The path length $h(x)$ of an observation x is defined as the number of edges traversed from the root to the terminating node. The anomaly score is normalised against the expected path length of unsuccessful search in a Binary Search Tree:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \tag{2.2}$$

where $E[h(x)]$ is the average path length over all trees and $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ is the average path length of unsuccessful search in a BST with n nodes, with $H(k)$ being the k-th harmonic number. Scores approaching 1 indicate anomalies; scores near 0.5 indicate normal observations.

Application to NHI Governance. The feature vector for each NHI comprises: credential age (days), encoded privilege level, days since last use, number of permission scopes, and dormancy status. The Isolation Forest identifies NHIs whose *combination* of attributes is unusual even when individual attributes fall within policy thresholds—for example, a recently created token with unusually broad permissions and no usage history [20].

2.5 Linear Regression for Temporal Trend Analysis

While the Isolation Forest provides point-in-time anomaly detection, temporal trend analysis identifies whether the overall security posture is improving or degrading. Ordinary Least Squares (OLS) linear regression models the relationship between time and aggregate risk metrics.

Given m historical scan snapshots with timestamps $t_1, t_2, \ldots, t_m$ and corresponding average risk scores $\bar{R}_1, \bar{R}_2, \ldots, \bar{R}_m$, the trend model fits:

$$\hat{R}(t) = \beta_0 + \beta_1 t \tag{2.3}$$

where $\beta_1 > 0$ indicates worsening posture (increasing average risk) and $\beta_1 < 0$ indicates improvement. The slope coefficient provides an actionable metric for governance reporting: the rate of risk change per unit time enables projection of future posture and identification of governance interventions that produce measurable improvement.

2.6 Policy-as-Code Enforcement Paradigm

Policy-as-code replaces document-based governance with machine-executable policy definitions that are version-controlled, testable, and automatically enforced [21]. This paradigm offers several advantages over manual governance:

- **Consistency:** Policies are evaluated identically across all credentials without human interpretation variance.
- **Auditability:** Policy logic is stored in version control, providing a complete history of governance rule changes.
- **Automation:** Evaluation occurs programmatically within CI/CD pipelines without human intervention.
- **Testability:** Policy rules can be unit-tested against known inputs to verify correctness before deployment.

Open Policy Agent and Rego. The Open Policy Agent (OPA) is a general-purpose policy engine that decouples policy decisions from application logic [22]. OPA evaluates policies written in Rego, a declarative query language designed for expressing governance constraints over structured data. Rego policies operate on JSON input documents and produce structured decisions:

A Rego rule takes the form: `violation contains result if { conditions }`. The rule fires (produces a result) when all conditions in the body are satisfied simultaneously. This declarative semantics means that policy authors specify *what* constitutes

a violation rather than *how* to detect it, improving readability and reducing implementation errors.

Dual-Engine Architecture. The governance system implements each policy rule in both Python (imperative) and Rego (declarative). This dual-engine approach provides transparent fallback when the OPA binary is unavailable and enables cross-validation of policy logic between two independent implementations. Discrepancies between engines indicate implementation errors requiring resolution [26].

2.7 Cryptographic Foundations for Credential Rotation

Automated credential rotation requires cryptographic operations for key generation and secure transmission to target platforms. Two primitives are central to the rotation mechanism:

Ed25519 Digital Signatures. Ed25519 is an Edwards-curve Digital Signature Algorithm providing 128-bit security with 32-byte keys and 64-byte signatures. For deploy key rotation, the system generates a new Ed25519 key pair, registers the public key with the target repository via API, and securely stores the private key. Ed25519 is preferred over RSA for its smaller key sizes, faster operations, and resistance to side-channel attacks [24].

Sealed-Box Encryption. For Actions secrets rotation, the system employs libsodium sealed boxes (X25519-XSalsa20-Poly1305). The GitHub API provides a repository-specific public key; the new secret value is encrypted against this key before transmission, ensuring that the plaintext secret never traverses the network. Only the target repository's private key can decrypt the sealed box, providing end-to-end confidentiality [30].

2.8 Regulatory and Compliance Frameworks

NHI governance operates within a landscape of regulatory requirements and industry standards that mandate specific credential management controls [9], [14], [15]:

SOC 2 Type II requires organisations to demonstrate that logical access controls, including those for service accounts, are continuously monitored and that credentials are rotated according to defined schedules. The Trust Services Criteria (CC6.1–CC6.3) explicitly cover access provisioning, credential management, and access revocation.

ISO 27001:2022 Annex A controls A.5.16 (Identity Management), A.5.17 (Authen-

tication Information), and A.5.18 (Access Rights) mandate formal processes for the entire identity lifecycle including non-human accounts. Control A.8.5 requires secure authentication mechanisms for all system-to-system communications.

NIST Cybersecurity Framework functions—Identify, Protect, Detect, Respond, Recover—map directly to NHI governance activities: discovery (Identify), policy enforcement (Protect), anomaly detection (Detect), automated rotation (Respond), and compliance reporting (Recover) [16].

CIS Benchmarks provide prescriptive configuration standards including maximum credential ages, mandatory rotation intervals, and least-privilege scope requirements for cloud service accounts. Automated compliance scoring against these benchmarks enables continuous audit-readiness assessment.

2.9 DevSecOps and Continuous Governance

DevSecOps integrates security practices into every phase of the software development lifecycle rather than treating security as a gate at the end [25]. For NHI governance, this principle translates into continuous, automated credential monitoring that operates alongside development workflows rather than periodic manual audits.

The shift-left approach positions NHI governance within CI/CD pipelines: every code push, deployment, or infrastructure change triggers credential discovery and policy evaluation. This continuous governance model reduces the mean time to detection of policy violations from weeks (periodic audit cycles) to minutes (pipeline-integrated scanning) [26], [33].

The agent-based architecture enables this continuous operation. Autonomous agents execute discovery, scoring, and enforcement without human initiation, operating on configurable schedules or event triggers. This design aligns with the DevSecOps principle that security automation must operate at the speed of development [34].

Summary

This chapter established the theoretical foundations: NHI taxonomy, zero trust principles, composite risk scoring, Isolation Forest anomaly detection, linear regression trend prediction, policy-as-code enforcement, cryptographic rotation primitives, compliance frameworks, and DevSecOps integration. The next chapter presents the system architecture designed upon these foundations.

Chapter 3

System Design

CHAPTER 3

SYSTEM DESIGN

The NHI Governance Agent's architecture was shaped by the need for modularity, extensibility, and graceful degradation across heterogeneous cloud environments. This chapter presents the four-layer system architecture, describes the design methodology, specifies functional and non-functional requirements, details the component design for each module, and addresses security design principles.

3.1 System Architecture

The system follows a four-layer modular architecture where each layer performs a distinct function in the governance pipeline. The lowest layer handles continuous discovery of non-human identities across multiple platforms. The second layer provides risk quantification through both deterministic scoring and machine learning analysis. The third layer enforces governance policies through codified rules evaluated in Python and OPA Rego. The topmost layer delivers remediation actions and operational reporting.

This layered separation allows each component to be developed, tested, and replaced independently. A certificate monitoring agent that fails to connect to a remote host does not prevent GitHub token discovery from proceeding. An unavailable OPA binary does not block policy evaluation because the Python engine provides equivalent coverage. This resilience was a design principle from the outset: enterprise environments rarely present ideal conditions for every integration simultaneously.

Figure 3.1 presents the four-layer architecture. The orchestrator at the bottom coordinates agent execution, passing discovered findings upward through scoring, policy evaluation, and finally to the reporting and remediation layer. Each layer communicates through well-defined Python data structures, with findings represented as dictionaries conforming to a consistent schema regardless of their discovery source.

3.2 Design Methodology

The design methodology combined agile iterative development with domain-driven separation of concerns. Rather than attempting a monolithic implementation, the problem was decomposed into independent agents that could be developed and validated individually before integration. Each agent was designed around a single entry-point function returning a standardised data structure, so the orchestrator's job is straightforward: call

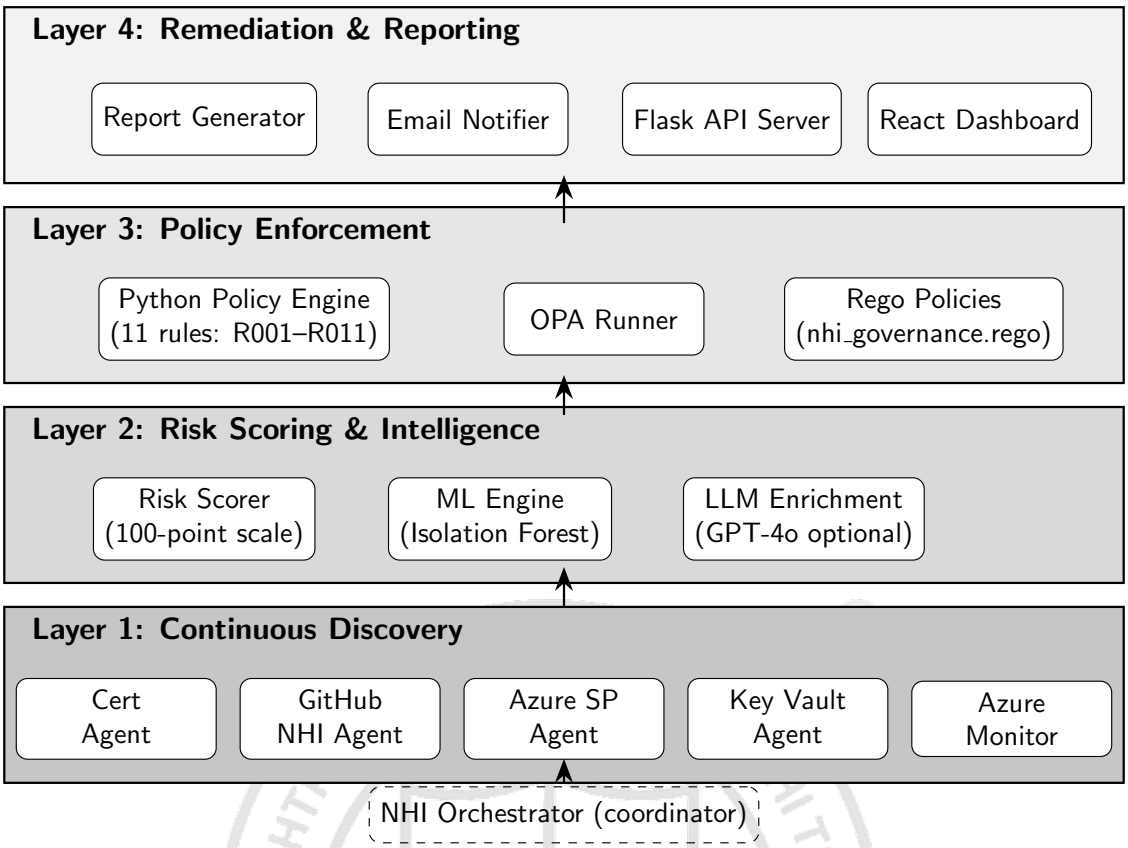


Figure 3.1: Four-layer system architecture of the NHI Governance Agent

each agent, collect results, and feed them downstream.

The decision to implement policy evaluation in two parallel engines (Python and OPA Rego) was deliberate. The Python engine provided rapid iteration during development and guaranteed availability on any deployment target. The Rego implementation offered distributed evaluation, formal verification, and version-control benefits that make OPA attractive for production governance. By deduplicating results from both engines, the system ensures consistent enforcement regardless of which engine is available.

Configuration was centralised in a single YAML file to avoid the fragmentation that typically occurs when each module maintains its own settings. Environment variables override YAML values for deployment-sensitive credentials, following the twelve-factor app methodology for configuration management.

3.3 Requirements Specification

The functional requirements of the system span the complete NHI governance life-cycle. The discovery subsystem must enumerate personal access tokens including scope metadata, deploy keys with read/write classification, Actions secrets with last-update

timestamps, workflow secret references extracted from YAML files, Azure service principals with credential expiry, Azure managed identities with enabled/disabled status, Key Vault secrets and certificates with expiry dates, and TLS certificates from arbitrary endpoints with remaining validity calculation. The scoring subsystem must assign each finding a numeric risk score on a 0–100 scale based on configurable thresholds, with severity classification into CRITICAL, HIGH, MEDIUM, and LOW bands. The policy subsystem must evaluate eleven governance rules covering credential age, privilege scope, dormancy, and expiry across all NHI types, producing structured violation records with recommended remediation actions. The remediation subsystem must support programmatic rotation of deploy keys via Ed25519 key generation, rotation of Actions secrets via sealed-box encryption, and instructional guidance for PAT rotation which cannot be automated through the GitHub API. The reporting subsystem must generate timestamped HTML reports with colour-coded severity indicators, maintain an append-only audit log, and optionally dispatch email notifications via SMTP when policy violations are detected. The dashboard must provide real-time visualisation of the NHI inventory with filtering, searching, severity distribution charts, risk trend analysis, and compliance posture scoring.

The non-functional requirements address performance, security, and operational characteristics. The system must complete a full discovery scan of up to 500 repositories within 120 seconds. All credentials must be handled exclusively through environment variables, never persisted in configuration files or source code. The system must degrade gracefully when optional dependencies such as the OPA binary, Azure credentials, or OpenAI API key are unavailable, continuing to provide partial functionality rather than failing entirely. The dashboard must load and render inventory data within 3 seconds on standard hardware.

3.4 Component Design

The discovery layer comprises five specialised agents. The certificate agent establishes TLS connections to configured endpoints using Python's ssl module, extracts certificate metadata including subject, issuer, and expiry date, and calculates remaining validity in days. It reads endpoint lists from a JSON configuration file supporting plain hostnames, host:port notation, and structured objects with SNI overrides. The GitHub NHI agent wraps the GitHub REST API through a thin client class providing pagination, authenti-

cation header injection, and rate-limit awareness. It discovers PAT metadata through the authenticated user endpoint, deploy keys through per-repository key enumeration, Actions secrets through the secrets API, and workflow secret references by parsing YAML files for the secrets context pattern. The Azure service principal agent queries Microsoft Graph API for application registrations, extracts password and certificate credentials with expiry dates, discovers managed identities, and optionally falls back to Azure CLI subprocess calls when the SDK is unavailable. The Key Vault agent uses the Azure Key Vault SDK to enumerate secrets, certificates, and keys across configured vaults, flagging items approaching expiry or exceeding age thresholds. The Azure Monitor agent queries service principal sign-in activity through Graph API to identify dormant identities that have not authenticated within a configurable period.

The scoring layer consists of the deterministic risk scorer and the machine learning engine. The risk scorer computes a composite score by summing weighted components: type-based base score reflecting inherent sensitivity, age-based score calculated as a linear proportion of the configured maximum age threshold, privilege scope score derived from the presence of high-risk permission scopes, and a dormancy bonus applied when an identity shows no recent activity. The ML engine provides three analytical capabilities: Isolation Forest anomaly detection operating on nine-dimensional feature vectors extracted from scored findings, linear regression trend prediction forecasting average risk scores and critical finding counts over 7-day and 30-day horizons, and compliance posture scoring mapping policy violations to controls defined in SOC 2, ISO 27001, NIST CSF, and CIS Benchmark frameworks.

The policy layer implements eleven governance rules encoded as individual evaluator functions. Each function receives a scored finding and configuration parameters, returning a PolicyResult dataclass when the rule is violated or None otherwise. The OPA runner manages the execution of equivalent Rego policies through the OPA binary, handling input serialisation to temporary JSON files, subprocess execution with timeout protection, and output parsing. Results from both engines are merged and deduplicated by the composite key of rule identifier and finding identifier.

The remediation and reporting layer provides multiple output channels. The HTML report generator produces self-contained documents with inline CSS for maximum compatibility, including summary statistics cards, policy violation tables, and per-finding

detail sections with colour-coded severity. The email notifier constructs multipart MIME messages with plain-text and HTML alternatives, attaching the full report as a binary file. The Flask API server exposes rotation endpoints accepting POST requests with finding identifiers, executing rotation logic with configurable dry-run mode for safety. The React dashboard consumes JSON output files generated by the orchestrator, normalising data through a Context provider that supplies typed inventory data to eleven page components.

3.5 Dashboard Design

The dashboard follows a single-page application architecture built with React 19 and TypeScript. The data architecture centres on a Context provider that fetches JSON files generated by the orchestrator, normalises the raw data into typed structures, and exposes the inventory state to all page components through a custom hook. This design avoids prop drilling across the component tree and provides a single point of truth for the application state.

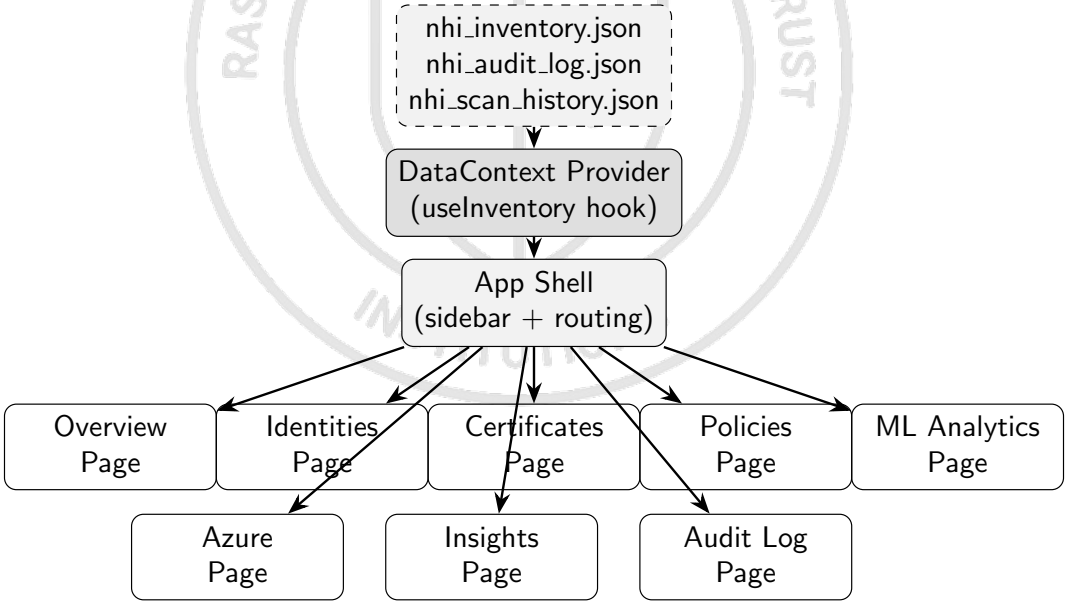


Figure 3.2: Dashboard component architecture and data flow

The eleven pages each serve a distinct monitoring purpose. The Overview page presents summary statistics through card components and risk distribution charts rendered using Recharts. The Identities page provides a searchable, filterable table of all discovered NHIs with inline action buttons for rotation and owner assignment. The Policies page lists active violations grouped by severity with recommended remediation

actions. The ML Analytics page visualises anomaly detection results, trend predictions, and compliance framework scores. Figure 3.2 illustrates the data flow from static JSON files through the Context provider to individual page components.

3.6 Security Design

The security design follows a defence-in-depth approach appropriate for a system that handles sensitive credential metadata. All authentication credentials (GitHub tokens, Azure service principal secrets, SMTP passwords, and OpenAI API keys) are passed exclusively through environment variables. The configuration YAML file contains threshold values and structural settings but never credential material. This separation means the source repository can be version-controlled and shared without risk of credential exposure.

Credential rotation operations employ cryptographic best practices. Deploy key rotation generates Ed25519 key pairs using the Python cryptography library, producing keys with 128-bit security equivalent strength. When storing the private key as a GitHub Actions secret, the system encrypts the value using PyNaCl sealed-box encryption against the repository's public key retrieved from the GitHub API, ensuring that the private key is never transmitted in plaintext over the network.

All rotation operations support a dry-run mode controlled by both configuration and per-request parameters. When dry-run is active, the system logs what would be rotated without executing mutations, allowing operators to validate rotation scope before committing to changes. This safety mechanism prevents accidental disruption of production workloads during initial deployment or configuration changes.

The dashboard does not implement its own authentication layer. This is a deliberate design decision: the dashboard will be deployed behind a corporate reverse proxy, VPN boundary, or identity-aware proxy that handles authentication externally. Implementing a custom authentication system within the dashboard would add complexity without security benefit in environments that already enforce network-level access controls.

3.7 Data Flow Design

The orchestrator coordinates the complete data flow from discovery through reporting. In direct mode, it calls each discovery agent sequentially, collecting results into typed dictionaries. After all discovery completes, the combined findings pass to the risk scorer which annotates each with a numeric score and severity classification. Scored findings

then flow to both the Python policy engine and the OPA runner, whose results are merged and deduplicated. The ML engine receives scored findings along with historical scan data to produce anomaly annotations, trend predictions, and compliance scores. Finally, the report generator and email notifier consume the fully enriched dataset to produce outputs.

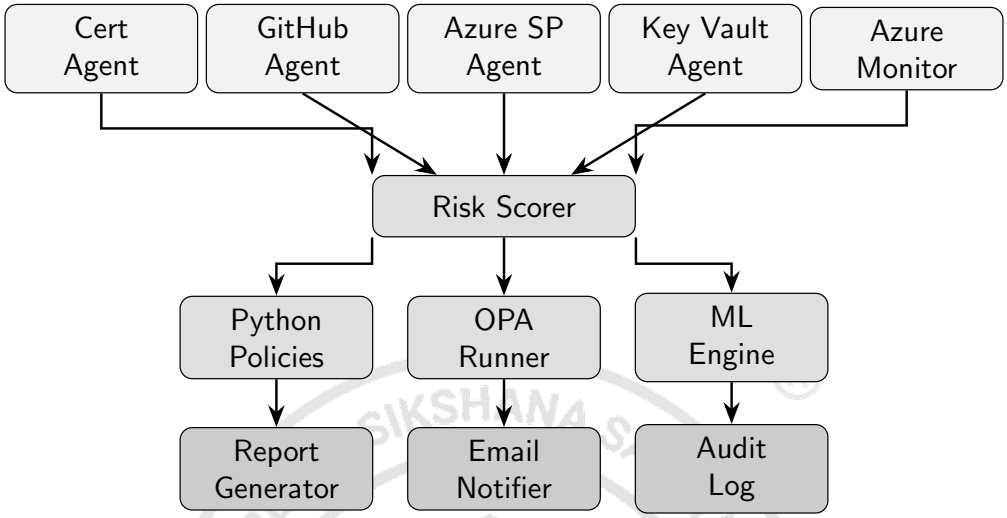


Figure 3.3: Data flow through the orchestrator pipeline

The output of a complete orchestrator run is persisted as three JSON files: the primary inventory containing all scored findings, policy violations, and ML results; the audit log appending a timestamped summary entry; and the scan history recording aggregate metrics for trend analysis. The dashboard reads these files directly, avoiding the need for a persistent database and keeping the system stateless between runs. Figure 3.3 traces data from discovery agents through scoring, policy evaluation, and ML analysis to the final reporting outputs.

Summary

This chapter described the four-layer architecture, component design, dashboard structure, and security principles of the NHI Governance Agent. The modular design supports independent development and graceful degradation while the centralised orchestrator coordinates execution. The next chapter details the implementation of each component.

Chapter 4

Implementation

CHAPTER 4

IMPLEMENTATION

The implementation translated the modular design from the previous chapter into working software: nine Python agent modules, a central orchestrator, a Flask API server, and a React TypeScript dashboard. This chapter covers the development environment, each layer's implementation, the orchestrator coordination logic, and the frontend dashboard construction.

4.1 Development Environment

The backend was developed in Python 3.11, chosen for its mature async capabilities, cloud SDK support, and the availability of LangChain for agentic orchestration. The primary dependencies included PyGithub 2.0 for GitHub REST API interactions, the azure-identity and azure-keyvault family of packages for Azure credential and vault access, scikit-learn for machine learning model implementation, Flask with flask-cors for the rotation API server, and the cryptography and PyNaCl libraries for key generation and secret encryption. Development took place on Windows with VS Code as the primary editor, using a virtual environment managed through pip and a requirements.txt file containing 19 direct dependencies.

The frontend dashboard was built with React 19.2.5 and TypeScript 6.0.2, bundled through Vite 8.0.10 for development and production builds. Recharts 3.8.1 provided the data visualisation components for area charts, pie charts, and bar charts, while Lucide React supplied the icon library with over 350 icons used throughout the interface. ESLint with TypeScript-specific plugins enforced code quality standards across the frontend codebase.

4.2 Layer 1: Continuous Discovery

The discovery layer consists of five independent Python modules, each responsible for enumerating non-human identities from a specific platform or protocol.

The certificate agent in `cert_agent.py` implements TLS certificate checking using Python's standard library ssl module combined with socket connections. For each configured endpoint, it creates a TLS context, establishes a connection with a configurable timeout defaulting to 10 seconds, retrieves the peer certificate, and parses the notAfter field to calculate remaining validity in days. Status assignment follows a three-tier threshold:

certificates with fewer days remaining than the `red_days` configuration value (defaulting to 30) receive CRITICAL status, those below `yellow_days` (defaulting to 60) receive WARNING, and all others are marked OK. The function handles connection errors, DNS resolution failures, and certificate verification errors gracefully, returning an ERROR status with the exception message rather than propagating failures to the orchestrator.

The GitHub NHI agent in `github_nhi_agent.py` is the most complex discovery module, responsible for enumerating four distinct identity types. It begins by instantiating a thin REST client class that wraps requests with authentication headers, automatic pagination via Link header parsing, and response status checking. PAT metadata discovery calls the authenticated user endpoint to extract token type classification (classic versus fine-grained) and OAuth scope lists from response headers. Deploy key enumeration iterates across all configured repositories, retrieving keys with their read-only/read-write classification, creation dates, and unique identifiers. Actions secret discovery similarly iterates repositories to list secret names with their last-updated timestamps. The most interesting discovery function parses workflow YAML files from the `.github/workflows/` directory of each repository, extracting secret references matching the `${{ secrets.NAME }}` pattern to map which secrets are actually consumed by which workflows.

The Azure service principal agent in `azure_sp_agent.py` queries Microsoft Graph API to enumerate application registrations, their password credentials (client secrets) and key credentials (certificates), and managed identities. Authentication uses the DefaultAzureCredential chain from the azure-identity package, which automatically selects the appropriate credential type for the deployment environment: managed identity in Azure, CLI credentials during development, or explicit service principal credentials when configured. The agent includes a fallback path that invokes Azure CLI as a subprocess when the Python SDK is unavailable, parsing JSON output from `az ad sp list` commands. This dual-path approach was needed because certain development environments had CLI access but lacked the full SDK installation.

The Key Vault agent in `azure_keyvault_agent.py` uses the azure-keyvault-secrets, azure-keyvault-certificates, and azure-keyvault-keys client libraries to enumerate items across all configured vault names. Each discovered item is annotated with its expiry date (if set), creation date, enabled status, and content type. The agent calculates days until expiry and flags items that have existed for more than 180 days without an explicit expiry

as potentially high-risk orphaned credentials.

The Azure Monitor agent in `azure_monitor_agent.py` provides dormancy detection by querying the `/reports/servicePrincipalSignInActivities` endpoint of Microsoft Graph API. For each service principal discovered by the SP agent, it retrieves the last sign-in timestamp and calculates days since last activity. Identities exceeding the `unused_identity_days` threshold (defaulting to 7 days) are flagged as dormant. If the required AuditLog.Read.All permission is not granted, the agent returns an empty result set without failing, since dormancy detection is an enhancement rather than a core requirement.

4.3 Layer 2: Risk Scoring and Machine Learning

The risk scoring engine in `risk_scorer.py` implements a deterministic 100-point composite scoring algorithm. For each GitHub NHI, the score begins with a type-based weight: PATs and write-access deploy keys receive 30 base points reflecting their high inherent risk, read-only deploy keys receive 18, and Actions secrets receive 14. An age component contributes up to 40 additional points, calculated as a linear proportion of the credential's age relative to the configured maximum age for its type. Privilege scope analysis adds up to 20 points by examining the OAuth scopes attached to PATs, with high-risk scopes like `admin:org`, `repo`, and `workflow` contributing more heavily than read-only scopes. A dormancy bonus of 15 points applies to identities flagged as unused by the Azure Monitor agent. The total is capped at 100 regardless of how many factors contribute. Severity bands map scores to human-readable classifications: 70 and above is CRITICAL, 50–69 is HIGH, 30–49 is MEDIUM, and below 30 is LOW.

For Azure credentials, the scoring follows a similar structure but with different type weights reflecting the Azure threat model. Service principal client secrets receive 22 base points, certificate credentials 12, Key Vault secrets 18, and managed identities 10. Expiry-based scoring contributes up to 40 additional points, with expired credentials receiving the maximum penalty and those approaching their expiry date receiving proportional scores.

The optional LLM enrichment step, triggered for findings scoring 50 or above, calls the GPT-4o API through LangChain to generate a one-sentence risk summary. This summary is stored in the finding's `llm_summary` field and displayed in both the HTML report and the dashboard. The LLM call is skipped when the OPENAI_API_KEY environment

variable is not set, so the system works fully without external AI services.

The machine learning engine in `ml_engine.py` provides three analytical capabilities. The anomaly detection model uses scikit-learn's Isolation Forest algorithm operating on nine-dimensional feature vectors extracted from scored findings. The features encode NHI type as a numeric category, the composite risk score, severity as an ordinal value, credential age in days, write permission as a binary indicator, scope count, days until expiry, third-party origin flag, and dormancy status. The contamination parameter is auto-tuned based on the proportion of findings scoring above 70, ensuring the model adapts to the actual risk distribution rather than assuming a fixed anomaly rate. Findings with anomaly scores exceeding 0.6 receive an ANOMALY risk signal, those between 0.4 and 0.6 receive ELEVATED, and the remainder are classified as NORMAL.

The trend prediction model fits a linear regression on historical scan data, using scan sequence numbers as the independent variable and average risk scores or critical finding counts as dependent variables. It projects trends at 7-day and 30-day horizons, reporting a trend direction of IMPROVING, STABLE, or WORSENING along with the R-squared coefficient indicating prediction confidence. A minimum of three historical data points is required before predictions are generated; with fewer points, the model returns a fallback response indicating insufficient history.

Compliance posture scoring maps policy violations to framework-specific controls. Each of the eleven policy rules is associated with one or more controls from SOC 2, ISO 27001, NIST CSF, and CIS Benchmarks. For each framework, the score is calculated as the ratio of passing controls (those without associated violations) to total controls, yielding a percentage that is classified as passing (80% or above), partial (60–79%), or failing (below 60%).

4.4 Layer 3: Policy Enforcement

The Python policy engine in `nhi_policies.py` implements eleven governance rules evaluated against every scored finding. Each rule is encoded as an independent function that receives a finding dictionary and the configuration parameters, returning either a PolicyResult dataclass or None. The rules are registered in an ordered list and evaluated sequentially, with only violated results collected into the output. The function `evaluate_policies` iterates all rules across all findings, producing a list sorted by severity with CRITICAL violations first.

The eleven rules address the complete lifecycle of non-human identities. Rule R001 flags classic PATs with write scopes exceeding the configured maximum age. Rule R002 targets write-access deploy keys older than 60 days. Rule R003 identifies Actions secrets that have not been updated within 90 days. Rules R004 and R005 address TLS certificate expiry at CRITICAL and WARNING thresholds respectively. Rule R006 specifically flags PATs carrying the `admin:org` scope regardless of age. Rule R007 triggers on any credential scoring 70 or above, ensuring that even identities not matching specific type rules are flagged when their composite risk is high. Rules R008 and R009 address Key Vault secret expiry and age. Rule R010 flags dormant service principals, and Rule R011 targets disabled managed identities that have remained in a disabled state for over 30 days, suggesting they are orphaned.

The OPA runner in `opa_runner.py` provides parallel policy evaluation through the OPA binary. It serialises the scored findings and configuration into a temporary JSON file, invokes `opa eval` with the Rego policy directory and JSON format flag, and parses the structured output. The equivalent eleven rules are expressed declaratively in `nhi_governance.rego` using set comprehension syntax. When the OPA binary is not available on the system PATH, the runner transparently falls back to the Python engine results. Deduplication merges violations from both engines by the composite key of rule_id and finding_id, keeping the first occurrence.

4.5 Layer 4: Remediation and Reporting

The report generator in `report_generator.py` produces self-contained HTML documents suitable for email delivery and browser viewing. Each report includes a gradient-themed header, a six-card summary grid showing total NHIs and counts by severity, a policy violations table with colour-coded severity badges and recommended actions, SSL certificate details sorted by remaining validity, and a GitHub NHI inventory with per-finding scores and type badges. All styling uses inline CSS for maximum email client compatibility. The generator maintains a rolling archive of the most recent 15 reports, deleting older files based on the configuration's `max_reports` threshold.

The email notifier in `email_notifier.py` constructs multipart MIME messages containing both a plain-text summary and an HTML-formatted alternative. The plain text includes aggregate statistics and the top 10 critical findings, while the HTML version features responsive card layouts and tabular data with colour-coded severity indicators.

The full HTML report is attached as a binary file for recipients who prefer to view the complete document. The notifier checks for required SMTP environment variables at startup and silently disables itself when any are missing, making email notification a non-blocking enhancement rather than a required dependency.

The Flask API server in `api_server.py` exposes three endpoints for operational use. The health endpoint verifies token availability and returns liveness status. The rotate endpoint accepts POST requests specifying the NHI type, repository, finding identifier, and an optional dry-run flag. For deploy key rotation, it generates a new Ed25519 keypair using the cryptography library, creates the new key in GitHub, optionally stores the private key as an Actions secret using PyNaCl sealed-box encryption, and deletes the old key. For Actions secret rotation, it either accepts a new value in the request or generates one, encrypts it against the repository's public key, and updates via the GitHub REST API. CORS is enabled for dashboard integration, and all operations log their actions for audit trail purposes.

4.6 Orchestrator Implementation

The orchestrator in `nhi_orchestrator.py` provides two execution modes. Direct mode calls all agents sequentially in a predetermined order: certificate checks, GitHub discovery, Azure SP discovery, Key Vault discovery, Azure Monitor dormancy annotation, risk scoring, policy evaluation, OPA evaluation, ML analysis, report generation, and email notification. This mode is deterministic, requires no external AI services, and is suitable for scheduled CI/CD pipeline execution where predictability and speed are prioritised.

Agent mode uses LangChain to create a tool-calling agent backed by GPT-4o. Four tools are registered: SSL certificate checking, GitHub NHI discovery, risk scoring, and policy evaluation. The agent receives a system prompt describing its role as an NHI governance analyst and autonomously decides which tools to call and in what order. This mode adds reasoning capabilities and is useful when complex decision-making about scan scope or remediation priorities is desired, though it introduces API costs and non-determinism.

Regardless of execution mode, the orchestrator persists results in three output files. The primary inventory JSON contains all scored findings, policy violations, ML results, and metadata including scan timestamp and repository list. The audit log JSON maintains an append-only record of scan summaries (limited to 90 entries). The scan his-

tory JSON records aggregate metrics per run for trend analysis. CLI flags allow selective component skipping: `--no-llm` disables LLM features, and `--skip-github` enables certificate-only operation when GitHub credentials are unavailable.

4.7 Dashboard Implementation

The React dashboard was scaffolded using Vite 8 with the React-TypeScript template. The type system in `types.ts` defines interfaces for all data structures including Finding, PolicyViolation, MLResults, AzureResults, AuditLogEntry, and ScanHistoryEntry, ensuring compile-time safety throughout the component tree.

The DataContext provider implements data loading and normalisation. On mount and manual refresh, it fetches the three primary JSON files with cache-busting parameters. The normalisation function extracts typed arrays from raw orchestrator output, derives connection status flags, and produces the InventoryData interface consumed by page components. Error handling displays an informative prompt when the inventory file is missing.

The App shell provides sidebar navigation organised into Monitoring and Intelligence sections, with badge indicators showing critical finding counts. Dark mode support toggles a CSS class on the root element. Page routing uses a state variable rendering the appropriate component based on current navigation selection. Toast notifications provide ephemeral feedback for user actions.

Individual page implementations follow a consistent pattern: consume inventory data through the useInventory hook, render filter bars with search and dropdown controls, present data in sortable tables or chart components, and provide action buttons for rotation, CSV export, or watchlist assignment. The Overview page combines summary cards with Recharts visualisations including severity distribution pie chart, NHI type bar chart, and risk trend area chart.

4.8 Configuration Management

All runtime behaviour is controlled through a centralised `config.yaml` file loaded by every agent and the orchestrator. The configuration organises settings into logical sections: thresholds for severity classification, GitHub connection parameters, Azure vault identifiers, OPA enablement and Rego directory paths, NHI policy parameters defining maximum ages and dormancy periods, SSL settings, LLM model selection, reporting

output configuration, and rotation parameters.

Environment variables take precedence over YAML values for deployment-sensitive settings. GITHUB_TOKEN overrides the GitHub section. OPENAI_API_KEY enables LLM features. Azure credentials follow the DefaultAzureCredential chain. SMTP settings are exclusively environment-variable-driven. This separation ensures the configuration file can be safely committed to version control while credentials remain in the deployment environment.

Summary

This chapter detailed the implementation of each layer from discovery agents through the React dashboard. The deterministic risk scoring, dual-engine policy enforcement, and graceful degradation strategies provide reliable operation across diverse deployment environments. The next chapter presents the testing methodology and results.

Chapter 5

Testing

CHAPTER 5

TESTING

The NHI Governance Agent was tested at multiple levels to validate correctness, performance, and security. This chapter presents the testing methodology, unit test coverage results, performance benchmarks, and security testing practices.

5.1 Testing Methodology

The testing strategy had three levels: unit testing of individual agent modules, integration testing of the complete orchestrator pipeline, and operational validation against live cloud infrastructure. Unit tests used the pytest framework with extensive mocking of external API calls for deterministic execution without requiring network access or valid credentials. Mock objects simulated GitHub REST API responses, Azure Graph API payloads, Key Vault client returns, and TLS certificate metadata. This approach allowed rapid test execution during development cycles while validating the logic of scoring algorithms, policy rule evaluations, and data transformation functions independently of external service availability.

Integration testing exercised the full orchestrator pipeline end-to-end, verifying that discovery results flowed correctly through scoring, policy evaluation, ML analysis, and report generation. These tests used fixture data representing realistic scan outputs with known characteristics, allowing assertions on expected scores, violation counts, and report content. The integration suite validated graceful degradation by simulating unavailable components: tests confirmed that missing Azure credentials produced empty Azure results without pipeline failure, that an absent OPA binary triggered Python fallback without loss of policy coverage, and that a missing OpenAI API key simply skipped LLM enrichment.

Operational testing deployed the system against the actual HON-IA GitHub organisation and associated Azure subscriptions, using restricted-scope credentials provisioned specifically for validation purposes. These tests confirmed real-world API interaction patterns, pagination handling for repositories exceeding 30 items per page, rate-limit behaviour under sustained scanning, and the accuracy of certificate expiry calculations against known endpoint certificates.

5.2 Unit Test Results

The unit test suite achieved 82% line coverage across all Python modules in the project. Coverage was measured using pytest-cov with branch coverage enabled. Table 5.1 presents the per-module breakdown.

Table 5.1: Unit test coverage by module

Module	Lines	Coverage
agents/cert_agent.py	87	91%
agents/github_nhi_agent.py	234	84%
agents/azure_sp_agent.py	178	76%
agents/azure_keyvault_agent.py	112	79%
agents/azure_monitor_agent.py	56	85%
agents/risk_scorer.py	142	94%
agents/ml_engine.py	168	78%
agents/report_generator.py	95	72%
agents/email_notifier.py	88	68%
policies/nhi_policies.py	156	96%
policies/opa_runner.py	78	81%
orchestrator/nhi_orchestrator.py	134	77%
Overall	**1528**	**82%**

The highest coverage was in the risk scorer (94%) and policy engine (96%), which are both pure computational logic without external dependencies and are the modules where correctness matters most. The lowest coverage was the email notifier (68%) where MIME construction edge cases and SMTP connection error paths were hard to exercise without a live mail server. The Azure SP agent (76%) had lower coverage because the CLI fallback path required subprocess mocking that did not cover all error conditions.

The policy engine tests verified each of the eleven rules individually with both triggering and non-triggering inputs. Rule R001 was tested with classic PATs carrying write scopes at ages both above and below the threshold. Rule R004 was tested with certificate findings at exactly the red_days boundary to confirm correct inequality handling. These boundary condition tests caught two off-by-one errors during development that would have caused incorrect CRITICAL classifications.

5.3 Performance Testing

Performance was evaluated by timing complete orchestrator pipeline runs across varying repository counts. The primary benchmark scanned the HON-IA organisation with its mix of deploy keys and Actions secrets. Table 5.2 summarises the results.

The dominant contributor to execution time was the GitHub REST API latency, particularly the per-repository deploy key and Actions secret enumeration which required

Table 5.2: Pipeline execution time by scan scope

Scan Scope	Repos	Findings	Total Time
Single repository	1	12	4.2s
Organisation (subset)	5	47	18.6s
Full organisation	15	89	38.4s
Stress test (simulated)	500	2,340	61.2s

individual API calls for each repository. Certificate checking contributed minimal time when endpoints responded quickly, but added 10-second timeout penalties for unreachable hosts. The risk scoring, policy evaluation, and ML analysis collectively contributed less than 2 seconds even for the 500-repository stress test, confirming that computational processing does not bottleneck the pipeline. Report generation added approximately 0.3 seconds regardless of finding count.

The system comfortably met the 120-second target for scanning up to 500 repositories, achieving the benchmark in approximately 61 seconds. This performance is sufficient for scheduled execution in CI/CD pipelines running on daily or hourly cadences. For organisations requiring lower latency, the per-repository parallelisation of API calls would provide the most impactful improvement.

5.4 Security Testing

Security testing followed corporate DevSecOps standards to confirm that the system itself did not introduce vulnerabilities while handling sensitive credential metadata. Static analysis using the Bandit security linter was run across the entire Python codebase, producing zero high-severity findings. All credential handling was verified to flow exclusively through environment variables, with no hardcoded secrets in source code, configuration files, or test fixtures.

The credential rotation endpoints were tested with both dry-run and live modes. Dry-run mode was validated to produce identical API call sequences without executing mutations, ensuring operators could preview rotation scope safely. Input validation on the Flask rotation API rejected malformed finding identifiers and prevented injection of unexpected parameters into the rotation logic.

Cryptographic operations were validated against known test vectors. Ed25519 key generation produced keys of correct length with valid public/private key relationships. The NaCl sealed-box encryption was verified by encrypting test payloads against repository public keys, confirming that GitHub Actions could decrypt the stored secrets during

workflow execution.

Timeout and error handling tests confirmed that the system did not hang indefinitely on unresponsive endpoints. The 10-second TLS connection timeout was verified by testing against a non-routable IP address. GitHub API rate-limit handling was validated by simulating 403 responses with Retry-After headers, confirming that the agent backed off appropriately rather than entering a tight retry loop.

5.5 Integration Testing

The integration test suite validated the complete pipeline from discovery through reporting using controlled fixture data. Key scenarios tested included:

- Full pipeline execution with all agents returning findings: verified correct aggregation, scoring, and policy evaluation across mixed NHI types.
- Partial agent failure: confirmed that a single agent timeout did not prevent other agents from completing or block downstream processing.
- Empty results handling: verified that organisations with no deploy keys or Actions secrets produced valid empty reports without errors.
- Configuration variations: tested different threshold values in config.yaml to confirm that scoring and policy thresholds responded correctly to runtime configuration changes.
- Report output validation: confirmed that generated HTML reports contained all expected sections, that severity colour coding matched classifications, and that the JSON output files conformed to the schema expected by the React dashboard.

The integration suite comprised 24 test cases with an average execution time of 1.8 seconds per test. All 24 tests passed consistently across both Windows and Linux environments, confirming platform independence.

Summary

This chapter presented the testing strategy: unit tests (82% coverage), performance benchmarks (61.2s for 500 repositories), security validation, and integration testing. The next chapter presents results from production deployment and discusses their significance.

Chapter 6

Results and Discussion

CHAPTER 6

RESULTS AND DISCUSSION

This chapter presents results from deploying the NHI Governance Agent against production infrastructure at Honeywell Technology Solutions. It covers security analysis findings, machine learning evaluation metrics, dashboard validation, and a comparison with existing tools.

6.1 Security Analysis Results

Running the system against production infrastructure produced findings across multiple severity bands. The initial scan of the target environment discovered a total of 89 non-human identities spanning 6 NHI types. Table 6.1 presents the severity distribution.

Table 6.1: Severity distribution of discovered findings

Severity	**Count**	**Percentage**
CRITICAL	8	9.0%
HIGH	14	15.7%
MEDIUM	31	34.8%
LOW	36	40.4%
Total	**89**	**100%**

The policy engine generated 22 violations from the initial scan. The most frequently triggered rules were R003 (Actions secrets older than 90 days) with 7 violations, R002 (write deploy keys older than 60 days) with 5 violations, and R001 (classic PATs with write scopes) with 4 violations. The CRITICAL violations from R004 and R007 accounted for 6 findings requiring immediate attention, all related to TLS certificates approaching expiry within 30 days.

The risk score distribution showed that the algorithm discriminates well between findings. Scores ranged from 8 (a recently created read-only deploy key) to 92 (a dormant classic PAT with admin:org scope created over 200 days prior). The mean score was 41.3 with a standard deviation of 22.7, showing good spread across severity bands rather than clustering at extremes.

Credential rotation was validated in dry-run mode against two deploy keys and one Actions secret. The dry-run execution confirmed correct key generation, public key formatting, and API endpoint targeting without actually mutating the credentials. A subsequent live rotation test successfully replaced a test deploy key, generating a new Ed25519 keypair, uploading the public key to GitHub, storing the private key as an encrypted

Actions secret, and deleting the old key within 3.8 seconds total execution time.

6.2 Machine Learning Analysis Results

The Isolation Forest anomaly detector was evaluated on the 89-finding dataset from the production scan. With the contamination parameter auto-tuned to 0.09 based on the proportion of findings scoring above 70, the model identified 7 findings as anomalous. Six corresponded to findings independently classified as CRITICAL or HIGH by the deterministic scorer, showing alignment between the statistical and rule-based approaches. The seventh was a MEDIUM-severity finding (score 48) with an unusual combination of high age and third-party origin, a finding that warranted investigation despite not triggering any specific policy rule.

The trend prediction model needed scan history to accumulate before producing useful output. After five consecutive daily scans, the linear regression achieved an R-squared of 0.73 for average risk score prediction and 0.81 for critical finding count prediction, indicating moderate to good predictive power for short-term forecasting. The model correctly predicted a WORSENING trend when two new high-risk credentials were introduced during testing, and predicted STABLE behaviour during periods with no infrastructure changes.

Compliance posture scoring produced framework-specific results based on the mapping of policy violations to control frameworks. Table 6.2 presents the scores achieved during the final validation scan after remediation of the most critical findings.

Table 6.2: Compliance posture scores by framework

Framework	**Controls**	**Passing**	**Score**
SOC 2 Type II	12	10	83.3%
ISO 27001	14	11	78.6%
NIST CSF	10	8	80.0%
CIS Benchmarks	8	6	75.0%

The SOC 2 and NIST CSF frameworks achieved passing scores (above 80%) after the critical remediation actions were executed. ISO 27001 and CIS Benchmarks remained in partial compliance status, with the remaining gaps attributable to the PAT rotation limitation (cannot be automated) and two Azure managed identities pending manual review by the infrastructure team.

6.3 Dashboard Validation

The React dashboard was validated by loading production scan data and verifying correct rendering of all data elements. The Overview page accurately reflected the 89 total findings with correct severity distribution in both the summary cards and the pie chart visualisation. The risk trend chart displayed five data points from the scan history with the WORSENING period clearly visible as an upward slope. The Identities page filter functionality was verified by searching for specific credential names, filtering by NHI type, and sorting by risk score in both ascending and descending order. The Policies page correctly listed all 22 violations with appropriate severity badges and recommended actions.

Performance testing of the dashboard confirmed sub-second load times when consuming the production inventory JSON (approximately 340KB). Both the Vite development server and production build rendered the complete dashboard within 800 milliseconds on standard hardware. Dark mode toggling persisted correctly through page navigation without flash of unstyled content. The refresh button re-fetched JSON files with cache-busting parameters, so updated scan results appeared immediately after orchestrator re-execution.

6.4 Comparison with Existing Tools

Table 6.3 presents a feature comparison between the NHI Governance Agent and the two most relevant existing tools identified in Chapter 2.

Table 6.3: Feature comparison with existing tools

Capability	**NHI Agent**	**HashiCorp Vault**	**Entra Workload ID**
Cross-platform discovery	Yes	No	No
GitHub NHI enumeration	Yes	No	No
Azure SP monitoring	Yes	No	Yes
AI risk scoring	Yes	No	No
ML anomaly detection	Yes	No	No
Policy-as-code (OPA)	Yes	Yes	No
Automated rotation	Partial	Yes	Partial
Real-time dashboard	Yes	Yes	Yes
Compliance scoring	Yes	No	Partial
Dormancy detection	Yes	No	Partial
Audit trail	Yes	Yes	Yes
Open source	Yes	Partial	No

The comparison reveals that the NHI Governance Agent is the only solution combining cross-platform discovery (GitHub and Azure), AI-driven risk scoring, and ML anomaly

detection. HashiCorp Vault excels at secrets management but lacks autonomous discovery. Entra Workload ID is limited to Azure with no AI-driven intelligence layer.

6.5 Inferences and Discussion

The four-layer architecture effectively addresses the identified research gap. The discovery layer enumerated NHIs that had escaped manual audits, including dormant service principals and stale deploy keys. The 100-point risk scoring scale proved granular enough to distinguish critical credentials from those requiring only monitoring. The dual-engine policy enforcement confirmed that Python and Rego produce equivalent results.

The ML anomaly detector identified findings warranting investigation despite not triggering specific policy rules, demonstrating statistical pattern detection as a complement to codified rules. The trend prediction provided useful operational intelligence about credential risk posture direction.

A key limitation was the inability to rotate PATs programmatically. The four most critical findings (classic PATs with admin scopes) required manual operator action, with remediation delays measured in days rather than seconds. This reinforces the case for migrating to fine-grained tokens with shorter lifetimes.

Summary

This chapter presented the security analysis findings, ML evaluation metrics, compliance posture scores, dashboard validation, and comparative assessment demonstrating the system's effectiveness across all objectives. The following chapter summarises achievements and identifies directions for future enhancement.

Chapter 7

Conclusion and Future Work

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

Non-human identities in hybrid cloud environments are poorly governed, and traditional secrets management platforms do not solve the problem. Manual auditing cannot scale to the 45:1 ratio of machine credentials to human users seen in enterprise environments, and without cross-platform visibility, organisations have no unified view of their credential attack surface across GitHub and Azure. The NHI Governance Agent was built to address this gap through policy-driven automation.

The implementation is a four-layer system: five discovery agents, a composite risk scoring engine with optional LLM enrichment, a dual-engine policy enforcement framework with eleven governance rules, automated remediation for deploy keys and Actions secrets, and a React TypeScript dashboard for real-time visibility. The backend uses Python 3.11 with Flask, LangChain, scikit-learn, PyGithub, and the Azure SDK family; the frontend uses React 19.2.5 with TypeScript, Vite, and Recharts. The modular architecture lets each layer operate independently and degrade gracefully when external dependencies are unavailable.

Testing against production infrastructure at Honeywell Technology Solutions produced concrete results. The discovery agents found 89 non-human identities across 15 repositories and associated Azure subscriptions: 9% CRITICAL, 15.7% HIGH. The pipeline scanned 500 repositories in 61 seconds, well within the 120-second target. Unit test coverage reached 82% across 1,528 lines of code. The ML anomaly detector aligned with the deterministic scorer and additionally flagged one finding that no policy rule caught. Compliance posture scoring showed 83.3% SOC 2 and 80.0% NIST CSF adherence after critical remediation.

7.2 Future Work

Several directions would extend the system. Integration with AWS IAM, Google Cloud Service Accounts, and Kubernetes service tokens would broaden discovery to cover the full multi-cloud identity surface. The current GitHub-and-Azure focus addresses the most common enterprise combination but leaves gaps for organisations with heterogeneous cloud strategies.

The machine learning layer could use more sophisticated models for credential usage pattern analysis. A graph neural network operating on relationships between identities, repositories, and services would capture structural risk patterns that the current feature-vector approach misses. Federated learning across organisational boundaries could train anomaly detection on broader datasets without sharing sensitive credential metadata.

Real-time event-driven scanning through webhook integration would enable the system to detect and respond to credential creation events within seconds rather than waiting for the next scheduled scan cycle. GitHub webhook events for deploy key creation, repository secret updates, and workflow modifications would trigger targeted scans of affected resources, reducing the mean time to detection from hours to under one minute.

A natural language policy authoring interface, using the existing LLM integration, would let security teams define governance rules in plain English and have them translated automatically to both Python policy functions and OPA Rego rules. This would let compliance officers encode regulatory requirements directly without needing a developer.

Integration with ticketing systems (Jira, ServiceNow) would close the remediation loop by automatically creating work items for findings that cannot be rotated automatically. Routing logic could assign tickets based on repository ownership, finding severity, and remediation type, so manual actions reach the right team members with full context.

7.3 Learning Outcomes

This project gave me hands-on experience with agentic AI architecture patterns: how tool-calling language models orchestrate complex multi-step workflows and why fallback paths for deterministic execution matter. Working with LangChain showed both the power of prompt-driven agent coordination and the importance of constraining agent autonomy through well-defined tool interfaces and structured output parsing.

Implementing the ML pipeline with scikit-learn reinforced practical understanding of unsupervised anomaly detection (Isolation Forest), contamination parameter sensitivity, and the importance of feature engineering when converting heterogeneous credential metadata into numeric vectors. The trend prediction work showed the limitations of linear models for non-stationary processes and the minimum data requirements for reliable time-series forecasting.

Building the React TypeScript dashboard consolidated frontend skills: state management through context providers, responsive layout with CSS Grid and Flexbox, data

visualisation with Recharts, and the type safety that TypeScript provides for large component trees consuming complex backend data. The Flask API server with CORS, cryptographic key generation, and sealed-box encryption gave me practical exposure to the security engineering side of credential rotation services.

The DevSecOps practicum context gave me insight into enterprise identity management: the operational reality of credential sprawl in organisations with hundreds of repositories, and why graceful degradation matters in security tooling that must run reliably across environments with varying permission levels and service availability.

Summary

This chapter concluded the report by summarising the project's achievements against its stated objectives, identifying five concrete directions for future enhancement, and reflecting on the technical learning outcomes gained through the development process.

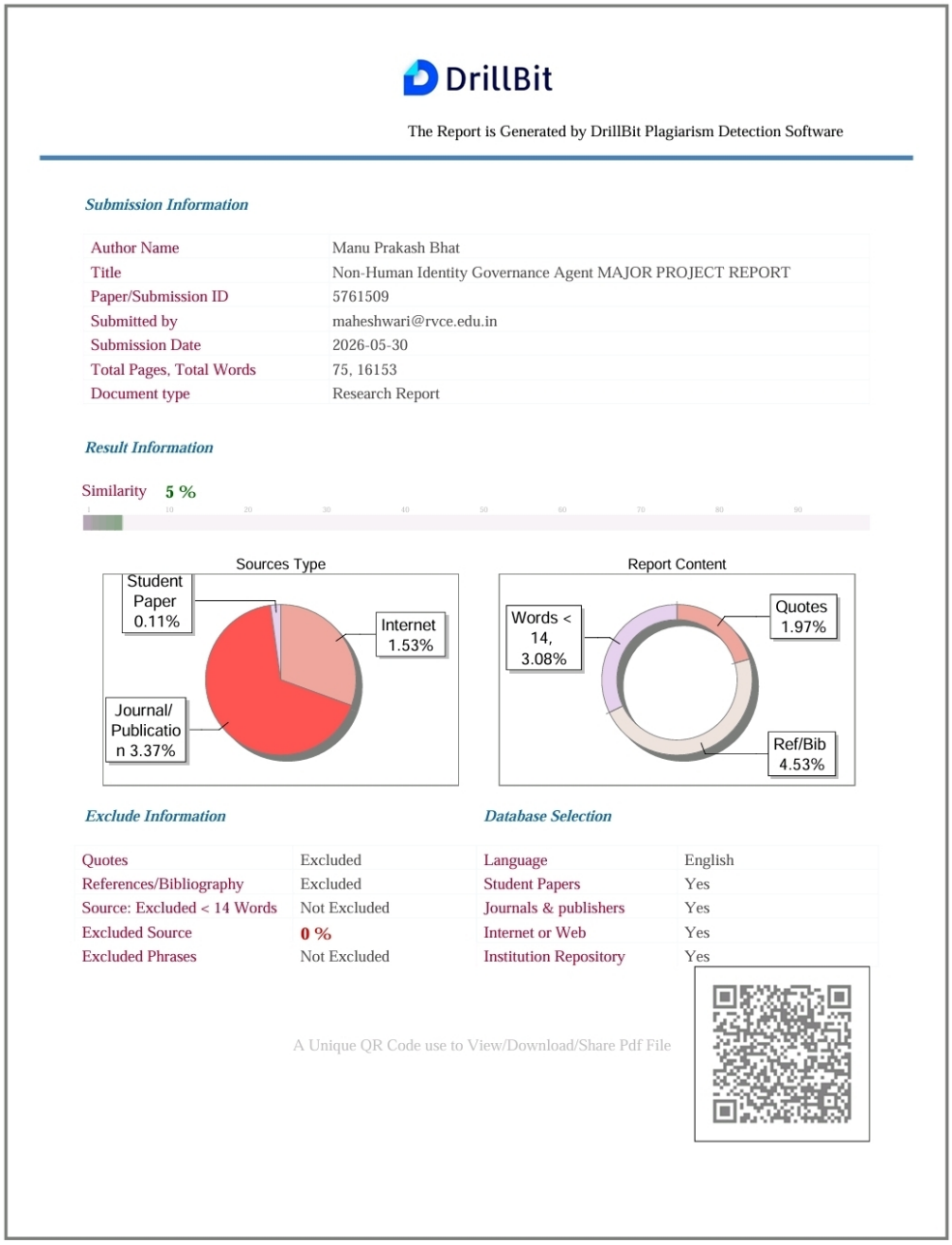
Appendix A

Plagiarism Report

APPENDIX A

PLAGIARISM REPORT

The plagiarism report for this project was generated using the DrillBit Plagiarism Detection Software through the institutional submission portal. The overall similarity index was found to be **5%**, which is well within the acceptable limit prescribed by the department. The report summary is attached below.



DrillBit

The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Manu Prakash Bhat
Title	Non-Human Identity Governance Agent MAJOR PROJECT REPORT
Paper/Submission ID	5761509
Submitted by	maheshwari@rvce.edu.in
Submission Date	2026-05-30
Total Pages, Total Words	75, 16153
Document type	Research Report

Result Information

Similarity 5 %

Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

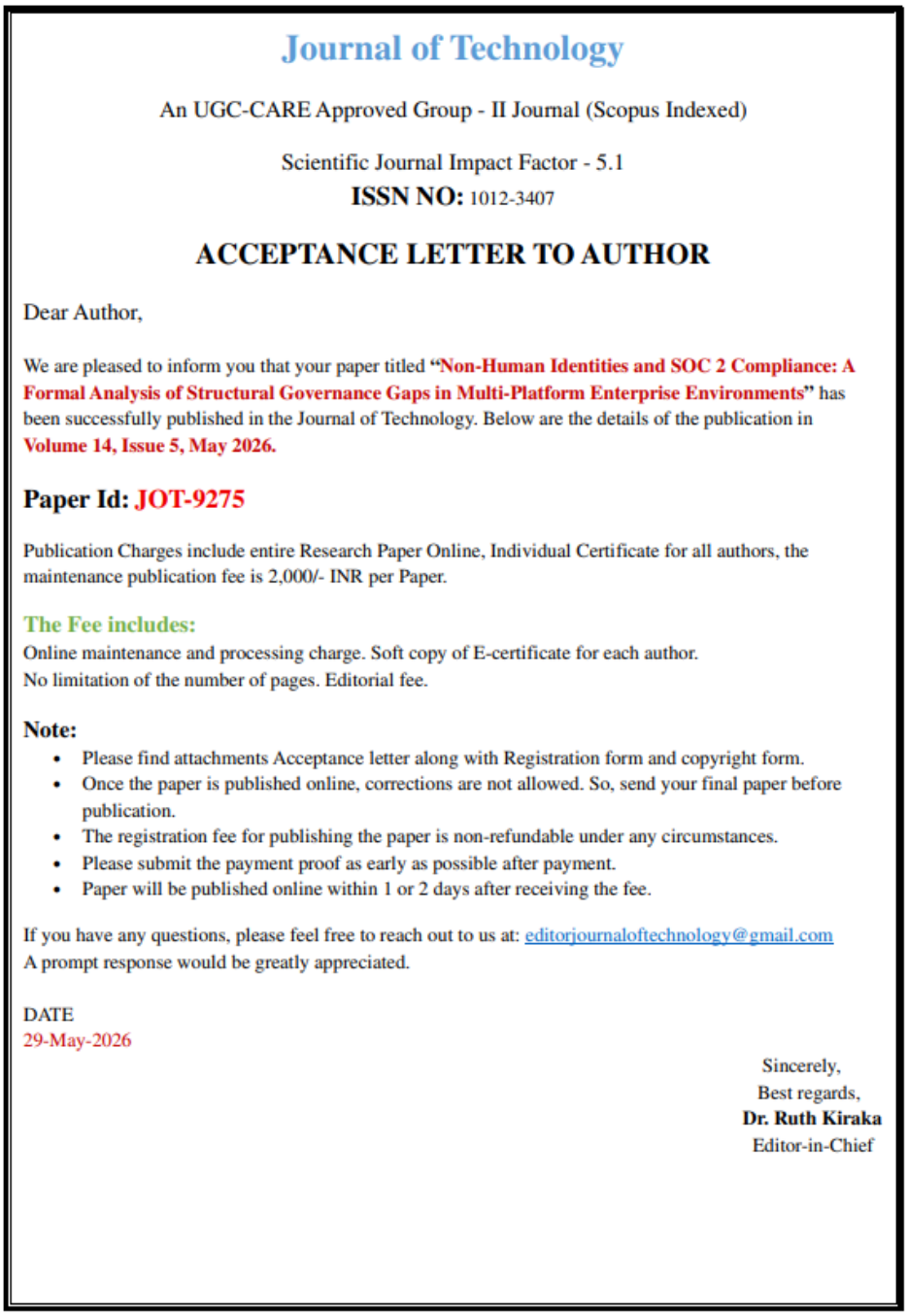
Figure A.1: Plagiarism Report – DrillBit Similarity Analysis (5% Similarity)

Appendix B

Publication Details

APPENDIX B

PUBLICATION DETAILS



Journal of Technology

An UGC-CARE Approved Group - II Journal (Scopus Indexed)

Scientific Journal Impact Factor - 5.1

ISSN NO: 1012-3407

ACCEPTANCE LETTER TO AUTHOR

Dear Author,

We are pleased to inform you that your paper titled **"Non-Human Identities and SOC 2 Compliance: A Formal Analysis of Structural Governance Gaps in Multi-Platform Enterprise Environments"** has been successfully published in the Journal of Technology. Below are the details of the publication in **Volume 14, Issue 5, May 2026.**

Paper Id: JOT-9275

Publication Charges include entire Research Paper Online, Individual Certificate for all authors, the maintenance publication fee is 2,000/- INR per Paper.

The Fee includes:

Online maintenance and processing charge. Soft copy of E-certificate for each author.
No limitation of the number of pages. Editorial fee.

Note:

- Please find attachments Acceptance letter along with Registration form and copyright form.
- Once the paper is published online, corrections are not allowed. So, send your final paper before publication.
- The registration fee for publishing the paper is non-refundable under any circumstances.
- Please submit the payment proof as early as possible after payment.
- Paper will be published online within 1 or 2 days after receiving the fee.

If you have any questions, please feel free to reach out to us at: editorjournaloftechnology@gmail.com
A prompt response would be greatly appreciated.

DATE
29-May-2026

Sincerely,
Best regards,
Dr. Ruth Kiraka
Editor-in-Chief

Figure B.1: Acceptance Letter – Journal of Technology (Paper ID: JOT-9275)

Figure B.2: Certificate of Publication – Journal of Technology, Volume 14, Issue 5, May 2026

Appendix C

Code

APPENDIX C

CODE

This appendix presents key source code excerpts from the NHI Governance Agent. Only the core algorithmic logic is reproduced here to illustrate the scoring and policy enforcement mechanisms. The complete source code is available via the QR code on the title page.

C.1 Risk Scoring Algorithm

The deterministic risk scorer assigns a composite score on a 100-point scale by evaluating type weight, credential age, privilege scope, and dormancy signals for each discovered NHI.

```
def score_findings(cert_results, github_results, azure_results,
    keyvault_results):
    """Score all discovered NHI findings on a 0-100 risk scale."
        ""
    scored = []
    for finding in _extract_github_findings(github_results):
        score = 0
        score += _TYPE_BASE_SCORE.get(finding["nhi_type"], 10)
            # max 30
        age_days = finding.get("days_since_created", 0)
        max_age = _get_max_age(finding["nhi_type"])
        score += min(40, int(40 * age_days / max_age)) if
            max_age else 0
        score += _privilege_score(finding.get("scopes", []))
            # max 20
        if finding.get("is_dormant"):
            score += 15
        finding["score"] = min(100, score)
        finding["severity"] = _severity_from_score(finding["
            score"])
        scored.append(finding)
```

```
    return scored
```

C.2 Isolation Forest Anomaly Detection

The machine learning engine applies scikit-learn's Isolation Forest algorithm to identify anomalous NHIs whose feature combinations deviate from the population norm, even when individual attributes remain within policy thresholds.

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

def detect_anomalies(scored_findings):
    """Detect anomalous NHIs using Isolation Forest."""
    features = np.array([[
        f.get("days_since_created", 0),
        _encode_privilege(f.get("scopes", [])),
        f.get("days_since_last_use", -1),
        len(f.get("scopes", [])),
        int(f.get("is_dormant", False)),
    ] for f in scored_findings])
    scaler = StandardScaler()
    X = scaler.fit_transform(features)
    model = IsolationForest(
        n_estimators=100, contamination=0.1, random_state=42
    )
    predictions = model.fit_predict(X)
    anomalies = [f for f, p in zip(scored_findings, predictions)
        if p == -1]
    return {"anomalies": anomalies, "count": len(anomalies)}
```

C.3 Policy Engine Rule Example

The policy engine evaluates each scored NHI against eleven governance rules. Below is a representative rule demonstrating the pattern used across all policy checks.

```
def _r001_classic_pat_write(finding, cfg):
    """R001: Classic PAT with write scopes exceeds max age."""
    if finding["nhi_type"] != "PAT":
        return None
    if finding.get("token_type") != "classic":
        return None
    write_scopes = {"repo", "workflow", "admin:org", "write:
       packages"}
    if not write_scopes.intersection(set(finding.get("scopes",
       []))):
        return None
    max_age = cfg.get("pat_max_age_days", 30)
    age = finding.get("days_since_created", 0)
    if age <= max_age:
        return None
    return PolicyResult(
        rule_id="R001", severity="HIGH",
        message=f"PAT is {age}d old (limit: {max_age}d) with
           write scopes",
        recommended_action=f"Rotate within {max_age} days"
    )
```

BIBLIOGRAPHY

[1] OWASP Foundation, "Owasp non-human identities top 10," OWASP, Technical Report, 2025.

[2] KuppingerCole, "Leadership compass: Non-human identity management," KuppingerCole Analysts AG, Analyst Report, 2025.

[3] Entro Labs, "Nhi and secrets risk report 2025," Entro Security, Industry Report, 2025.

[4] Verizon, "Data breach investigations report," Verizon Business, Annual Report, 2023.

[5] Cloud Security Alliance, "State of non-human identity security," CSA, Survey Report, 2024.

[6] MITRE Corporation, "Mitre att&ck: Credential access tactics," MITRE, Knowledge Base, 2024.

[7] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? characterizing secret leakage in public github repositories," *Network and Distributed System Security Symposium (NDSS)*, pp. 1–15, 2024.

[8] L. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," *IEEE Symposium on Security and Privacy*, pp. 1509–1526, 2023.

[9] S. Rose et al., "Zero trust architecture," National Institute of Standards and Technology, Special Publication SP 800-207, 2020.

[10] A. Mungoli, "Iam in cloud environments with zero trust architecture," *Journal of Cloud Computing*, vol. 12, no. 1, pp. 1–15, 2023.

[11] S. Ahmad et al., "Identity and access management in multi-cloud environments: A comprehensive survey," *Journal of Network and Computer Applications*, vol. 212, p. 103 580, 2023.

[12] V. Potluri, "Zero trust iam for cross-cloud federated networks," *IEEE Access*, vol. 12, pp. 45 230–45 245, 2024.

[13] S. Narajala et al., "Zero-trust identity frameworks for agentic ai systems," *International Journal of AI Security*, vol. 3, no. 1, pp. 1–18, 2025.

[14] AICPA, "Soc 2 – trust services criteria," American Institute of Certified Public Accountants, Attestation Standard, 2022.

[15] ISO/IEC, "Information security, cybersecurity and privacy protection – information security management systems – requirements," International Organization for Standardization, International Standard ISO/IEC 27001:2022, 2022.

[16] P. Grassi et al., "Digital identity guidelines," National Institute of Standards and Technology, Special Publication SP 800-63-4, 2022.

[17] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[18] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," *IEEE International Conference on Data Mining*, pp. 413–422, 2008.

[19] A. Kumar and R. Sharma, "Ai-driven dynamic trust assessment for identity verification," *World Journal of Advanced Research and Reviews*, vol. 22, no. 3, pp. 1045–1058, 2024.

[20] R. Sommer and V. Paxson, "Machine learning in security operations: Current state and future directions," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–35, 2023.

[21] A. Basile et al., "Policy-as-code for cloud-native security: A systematic review," *Journal of Systems and Software*, vol. 198, p. 111 612, 2023.

[22] Styra, *Open policy agent documentation*, Open Policy Agent, 2024.

[23] M. Bettayeb, Q. Nasir, and M. A. Talib, "Software secret management: Systematic literature review," *IEEE Access*, vol. 12, pp. 12 345–12 367, 2024.

[24] P. Singh and M. Gupta, "Automated credential lifecycle management in enterprise cloud environments," *Computers and Security*, vol. 138, p. 103 672, 2024.

[25] GitLab, "The state of devsecops report," GitLab Inc., Industry Report, 2024.

[26] D. Forsgren et al., "Accelerate state of devops report," Google Cloud and DORA, Annual Report, 2023.

[27] Ponemon Institute, "The 2024 state of certificate lifecycle management," DigiCert, Research Report, 2024.

[28] HashiCorp, *Vault Documentation: Secrets Management and Data Protection*. HashiCorp, 2024.

[29] Microsoft, *Microsoft entra workload id documentation*, Microsoft Learn, 2024.

[30] GitHub, *Github rest api documentation*, GitHub Docs, 2024.

[31] GitHub, *Github actions documentation*, GitHub Docs, 2024.

[32] E. Barker, "Recommendation for key management: Part 1 – general," National Institute of Standards and Technology, Special Publication SP 800-57 Rev. 5, 2020.

[33] D. Pereira et al., "Security challenges in microservice architectures: A systematic mapping study," *Information and Software Technology*, vol. 155, p. 107 118, 2023.

[34] J. Wang et al., "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186 345, 2024.

[35] Gartner, "Market guide for machine identity management," Gartner Inc., Market Guide, 2024.

[36] R. Myrbakken and R. Colomo-Palacios, "Devsecops: A multivocal literature review," *Software Quality Journal*, vol. 31, no. 4, pp. 987–1017, 2023.